CLIO Search: Agentic Scientific Data Discovery with Intelligent Orchestration

Author A and Author B
Department of Computer Science
University Name, City, Country
{authora, authorb}@university.edu

***Abstract*—Scientific data discovery requires search that acts, not just retrieves. Data is heterogeneous in format and expression, varies in the quality and quantity of metadata, and is scattered across storage systems. A human scientist might compensate naturally—converting units mentally, inferring from partial clues, checking secondary filesystems—but human effort does not scale to the billions of objects produced by modern science. No retrieval pipeline replicates this reasoning, because the required operations are not just arithmetic or navigational, but inferential.**

We present CLIO Search, a framework for agentic scientific discovery that implements intelligent orchestration, science-aware operators, federated connectors, and metadata-adaptive discovery. The system follows a four-stage pipeline: *Discover* (corpus profiling to learn what data exists and what metadata is available), *Reason* (adaptive branch selection and namespace routing based on corpus characteristics), *Transform & Search* (dimensional conversion across SI prefixes, formula normalization, and hybrid lexical-vector retrieval), and *Execute* (federated data access across six storage backends). A composable unit registry based on dimensional analysis covers 46 units across 12 physical domains. The agentic orchestration layer profiles each namespace before querying, selects only the retrieval branches that can contribute results, routes multi-namespace queries to the most relevant sources first, and tracks token efficiency across LLM-assisted query refinement hops.

We evaluate on two benchmarks with 80 queries. On cross-unit queries, CLIO Search achieves 0.99 P@5 where baselines score 0.40. On real-world NOAA observatory data, precision improves by 87% over state-of-the-art solutions. An agentic ablation study shows that corpus profiling achieves 95.8% branch selection efficiency, and metadata-adaptive strategy selection correctly identifies corpus characteristics for intelligent search orchestration. All code and benchmarks are open-source.

I. Introduction

Scientific computing generates data at unprecedented scale—a single exascale job on Frontier can produce 80 PB of simulation output—yet an estimated 60–80% of scientific data is never reused after creation, not because it is valueless, but because it is unsearchable. When a scientist cannot find that a 200 kPa combustion simulation already exists—stored as "200000 Pa" on a different filesystem—they re-run it, wasting allocations on machines funded by the DOE ASCR program's $1 billion annual budget. Unit confusion has real-world consequences beyond wasted compute: NASA's $327M Mars Climate Orbiter (pound-seconds vs. newton-seconds) and Air Canada's Gimli Glider incident (kg vs. lb fuel loading) are well-documented disasters. In healthcare, medication errors from unit confusion (mL vs. teaspoons, mg vs. grains) contribute to an estimated 7,000–9,000 deaths per year in the US alone. The FAIR data principles now mandated by NSF, NIH, and DOE require that data be *Findable*—yet automated FAIR assessments show that no major repository achieves even 50% compliance on reusability, with unit heterogeneity a systematic barrier. Meanwhile, pressure alone is expressed in at least six incompatible units across disciplines—Pa, hPa, bar, atm, psi, and torr—making cross-disciplinary discovery impossible without arithmetic conversion. A recent NAACL 2025 evaluation confirms that standard RAG pipelines systematically fail on numerical queries, with traditional search outperforming retrieval-augmented generation on quantitative content [1]. The problem is clear: a query for pressures between 190–360 kPa should match "200000 Pa" in an HDF5 attribute and "0.2 MPa" in an S3 dataset. These describe identical pressure. Every retrieval system treats them as unrelated strings.

This crisis undermines reproducibility—Nature's survey found over 70% of researchers have failed to reproduce others' experiments [2]—and is intensifying as AI agents enter scientific workflows. Coscientist [3] autonomously executes wet-lab synthesis; AI Scientist v2 [4] generates research end-to-end; autonomous agents at ORNL orchestrate experiments across facilities [5]. Yet ScienceAgentBench [6] reports only 32.4% end-to-end success, with failures traced to data handling, not reasoning. A comprehensive survey [7] identifies data management as explicitly underexplored. These agents can reason about science—they cannot find the data they need.

The root cause is structural: retrieval systems treat scientific content as plain text. Three compounding failures explain why.

Dimensional blindness. Scientific measurements are routinely expressed in different unit prefixes. String-based quantity normalization [8] maps unit names to canonical strings, but "kilopascal" and "pascal" remain different strings—no string operation bridges the SI prefix gap. Learned embeddings such as CONE [9] encode numbers and units in a shared vector space, but the Numeracy Gap study [10] evaluates 13 embedding models and finds 0.54 binary retrieval accuracy on numerical content—effectively random. NC-Retriever [11] reports 16.3% dense retrieval accuracy on numeric constraint queries. The fundamental problem is that none of these approaches performs the required arithmetic: $200 \times 1000 = 200{,}000$.

Formula opacity. "$F = ma$," "$F = m \cdot a$," and "$ma = F$" encode the same physical law but produce distinct token se-

quences. Math-aware search systems [12], [13] match equation structure in LaTeX markup but cannot connect a matched formula to the physical measurements it governs. To our knowledge, no retrieval system combines formula normalization with quantity-aware dimensional search.

Storage fragmentation. HPC data spans parallel filesystems, S3 object stores, vector databases, and binary formats such as HDF5 and NetCDF. To our knowledge, no system provides unified search with science-aware operators across these backends. RAGRoute [14] federates generic text search. PROV-IO+ [15] captures HDF5 provenance with under 3.5% overhead but exposes no search interface.

These failures compound. The motivating query—"find experiments where pressure was between 190–360 kPa across all storage"—requires dimensional conversion across SI prefixes, federated search across heterogeneous backends, and potentially formula context linking pressure to related variables. To our knowledge, no existing system addresses even two of these three requirements simultaneously.

We argue that these failures share a common root cause: retrieval pipelines lack a mechanism for incorporating domain knowledge as executable, verifiable computation. BM25 matches tokens. Dense models match embeddings. Neither can encode the fact that $5\ \mathrm{km} = 5000\ \mathrm{m}$—an algebraic invariant, not a statistical similarity. Unit conversion itself is well-established (UDUNITS-2 [16], GNU Units), but these tools operate *offline*—a scientist must already know which files to convert. The unsolved problem is bringing arithmetic conversion *inside* the retrieval pipeline so that the search itself bridges unit heterogeneity. We introduce *science-aware operators*: deterministic, pluggable retrieval primitives that encode domain knowledge as explicit arithmetic and symbolic transformations, executing as first-class branches alongside standard BM25 and dense vector search. The architecture is general—any domain where correctness is non-negotiable (chemical identifiers, genomic coordinates, engineering tolerances) can be served by operators of this class. We instantiate the framework for scientific measurement retrieval and test it through five contributions:

1) **Dimensional-conversion retrieval.** We canonicalize physical quantities via a composable unit registry based on dimensional analysis—each unit is a dimension vector over the 7 SI base dimensions plus a scale factor. The registry currently covers 46 units across 12 domains (pressure, temperature, energy, frequency, radioactivity, etc.), with correctness guaranteed by construction. Adding a new unit requires one configuration entry, not code changes.
2) **Formula normalization.** We normalize mathematical expressions across whitespace, superscript, side-swap, and factor-reordering variants, integrated with dimensional operators as parallel retrieval branches.
3) **Federated multi-namespace search.** A connector architecture supports unified retrieval across filesystem, S3, Qdrant, Neo4j, HDF5, and NetCDF backends with per-backend capability negotiation and incremental indexing.
4) **HDF5/NetCDF metadata indexing.** Dedicated connectors extract measurements from scientific file formats and feed them through the same SI canonicalization pipeline, enabling cross-format dimensional search.
5) **Agentic orchestration with metadata-adaptive discovery.** A corpus profiling layer inspects indexed namespaces before querying, enabling intelligent branch selection (95.8% efficiency), metadata-adaptive strategy selection, and namespace routing. An LLM-driven query rewriting loop iteratively refines searches using expand, narrow, and pivot strategies, with token efficiency tracking. A deterministic fallback achieves 96% of LLM-assisted quality.

The remainder of this paper is organized as follows. Section 2 surveys hybrid retrieval, the numeracy crisis, quantity-aware retrieval, and scientific data discovery. Section 3 presents the system design including the agentic orchestration layer. Section 4 describes the implementation. Section 5 evaluates across dimensional conversion, formula matching, federated search, agentic orchestration, and ablation. Section 6 concludes.

II. Background and Related Work

The gap between what retrieval systems deliver and what scientific data discovery demands spans four areas: the hybrid retrieval foundations we build upon, the numeracy crisis that motivates explicit arithmetic operators, quantity-aware retrieval systems that attempt but fail to solve cross-unit matching, and scientific data discovery in HPC.

A. Hybrid Retrieval Foundations

Modern retrieval combines lexical and dense signals. BM25 [17] remains the strongest lexical baseline; dense retrieval via learned bi-encoders [18] captures semantic similarity that keyword matching misses. Bruch et al. [19] show that a learned convex combination consistently outperforms reciprocal rank fusion (RRF) on TREC benchmarks, establishing the analytic framework we adopt. At the agentic frontier, A-RAG [20] introduces hierarchical retrieval interfaces that let LLM agents compose retrieval strategies dynamically.

We build *on* hybrid retrieval, not beside it. Our science-aware operators execute as parallel branches alongside BM25 and dense search within the same fusion pipeline, adding domain-specific recall without replacing any existing component.

B. The Numerical Reasoning Crisis

Numbers are fundamentally broken in both retrieval systems and large language models. The Numeracy Gap study [10] evaluates 13 embedding models on numerical content and reports 0.54 binary retrieval accuracy—barely above random chance. NC-Retriever [11] finds that dense retrieval achieves only 16.3% accuracy on queries with numeric constraints, even when the constraint is stated explicitly in both query

and document. The root cause is architectural: tokenizers encode numbers per-digit rather than by magnitude, so "200" and "200000" share no meaningful representation [21]. GSM-Symbolic [22] delivers the sharpest result: changing only the numeric values in grade-school math problems while preserving problem structure causes a 65% accuracy drop, proving that LLMs pattern-match on surface numerals rather than reason about magnitude.

This is not merely a unit problem. Numbers themselves—the substrate of all scientific measurement—are broken in every retrieval and language model. Scientific data retrieval, where matching requires comparing magnitudes across units, is especially vulnerable.

C. Quantity-Aware Retrieval

Three lineages attempt to make retrieval quantity-aware. None succeeds at arithmetic conversion.

String normalization. QFinder [23] extends Elasticsearch with quantity-aware ranking by extracting and normalizing numeric expressions. CQE [24] advances this with dependency parsing over a 531-unit dictionary, handling complex quantity expressions. Numbers Matter! [8] builds a full retrieval pipeline with quantity-aware indexing and evaluation on FinQuant and MedQuant benchmarks. The fundamental limitation persists: string normalization cannot cross SI prefixes. "kilopascal" and "pascal" remain distinct strings after normalization, so a query for "200 kPa" never matches a document containing "200000 Pa."

Learned embeddings. CONE [9] trains a transformer to embed number–unit pairs into a shared vector space, achieving a 25% Recall@10 gain on quantity-bearing queries. NC-Retriever [11] applies contrastive learning to numeric constraint matching. These approaches are probabilistic: a model may learn that 200 kPa is *similar* to 200000 Pa, but provides no correctness guarantee. Weller et al. [25] prove theoretical upper bounds on embedding-based retrieval, showing that fixed-dimensional embeddings cannot faithfully preserve all distance relationships—a result that undercuts learned approaches to exact quantity matching.

Math-aware search. Approach0 [12] matches equation structure via operator trees. SSEmb [13] learns semantic similarity over mathematical expressions. These systems match formula *structure* but are disconnected from physical measurements—they cannot link $F = ma$ to a document reporting force in newtons.

Offline conversion tools. Unit conversion itself is well-established: UDUNITS-2 [16] and GNU Units convert data values during preprocessing, but they do not operate at retrieval time, do not index measurements, and expose no search interface—a scientist must already know which files to convert.

Gap. To our knowledge, no existing *retrieval system* integrates arithmetic SI conversion as a first-class retrieval operator. Offline tools require locating data first; string normalization and learned embeddings operate within retrieval but cannot perform arithmetic. No system combines quantity-aware retrieval with formula normalization or federated search across HPC backends.

D. Scientific Data Discovery in HPC

Paper search. OpenScholar [26] indexes 45 million open-access papers and achieves GPT-4o-level citation quality. HiPerRAG [27] scales RAG to 3.6 million articles across DOE supercomputers Polaris, Sunspot, and Frontier. Both search *papers*—not experimental data, simulation outputs, or HDF5 files.

Data search. Recent systems do search scientific data. PANGAEA-GPT [28] deploys hierarchical multi-agent search over 400,000+ geoscientific datasets with data-type-aware routing and unit scale validation, achieving 8.14/10 relevance versus a 2.87/10 baseline. LLM-Find [29] uses LLM-generated code to retrieve geospatial data from heterogeneous sources with 80–90% success. The distinction is temporal: PANGAEA-GPT validates unit scales *post-hoc* (after retrieval); we convert *at retrieval time* (during indexing and query processing). LLM-Find generates access code per source; we provide reusable retrieval operators.

HPC data management. Datum [30] catalogs scientific data at INL but offers no content search. BRINDEXER [31] indexes POSIX metadata on Lustre with 69% query improvement—metadata only, no content. PROV-IO+ [15] captures HDF5 I/O provenance with under 3.5% overhead but exposes no search interface. HDF Clinic [32] proposes a knowledge graph over HDF5 but remains unimplemented.

Agentic retrieval. Context-1 [33] trains a 20B-token agentic search agent; PRISM [34] decomposes retrieval into precision/recall subagents; Agentic-R [35] co-trains retrievers with answer-correctness rewards (10–13% multi-hop gains). CLADD [36] and SciAgent [37] add domain knowledge operators for drug discovery and scientific workflows. These systems advance orchestration and learned retrieval. To our knowledge, none incorporates arithmetic dimensional conversion or formula normalization into the retrieval pipeline. Our work is complementary: PRISM and Agentic-R could be layered on top of our science-aware operators.

Table I summarizes the landscape. Individual capabilities exist in isolation—UDUNITS converts units offline, PANGAEA-GPT validates scales post-hoc, OpenScholar searches papers at scale—but to our knowledge, no prior *retrieval system* integrates arithmetic SI conversion as a first-class retrieval operator, combines it with formula normalization, or operates across federated HPC backends with per-connector capability negotiation. We honestly mark capabilities we lack—we do not index papers. PANGAEA-GPT receives partial credit for quantity extraction (unit scale validation, but post-hoc rather than at retrieval time).

III. System Design

To address the three failures identified in Section 1, we design CLIO Search around two central principles: science-aware operators must execute as first-class retrieval branches,

TABLE I
CAPABILITY COMPARISON ACROSS RETRIEVAL AND SCIENTIFIC DATA SYSTEMS. ✓ = SUPPORTED, ✗ = NOT SUPPORTED, ○ = PARTIAL.

	Our Contributions					General Capabilities				
System	SI Conversion	Formula Norm.	Sci. Branches	Federated	HDF5/NetCDF	Agentic	Qty. Extract.	Paper Search	Data Search	LLM Integr.
Ours (clio)	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
PANGAEA-GPT	✗	✗	✗	✗	✗	✓	○	✗	✓	✓
OpenScholar	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
HiPerRAG	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
Numbers Matter!	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
CONE	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
CLADD	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓
SciAgent	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓
Context-1	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓

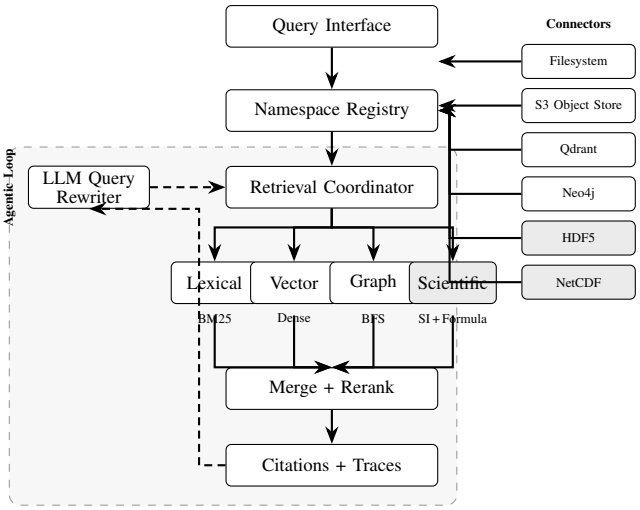

Fig. 1. System architecture of CLIO-AGENTIC-SEARCH. Science-aware components (shaded) execute alongside standard retrieval branches. The agentic loop iteratively refines queries via LLM rewriting.

and an agentic orchestration layer must reason about *what* to search before executing. The architecture comprises a staged pipeline with four parallel branches (Sec. 3.1), dimensional conversion operators that perform explicit SI arithmetic (Sec. 3.2), deterministic formula normalization (Sec. 3.3), a hybrid fusion strategy (Sec. 3.4), federated multi-namespace search (Sec. 3.5), and an agentic orchestration layer with corpus profiling, adaptive branch selection, and multi-hop refinement (Sec. 3.6).

A. Architecture Overview

The system is a staged pipeline with four parallel retrieval branches (Fig. 1). A query enters through the Query Interface (FastAPI REST endpoint or CLI), which parses the raw query string, extracts typed scientific constraints (numeric ranges, unit filters, formula patterns), and forwards a structured query object to the Namespace Registry. The registry resolves which namespaces—each backed by one of six connector types—are eligible for the query based on capability negotiation (Sec. 3.5).

The Retrieval Coordinator fans the query out to four parallel branches: lexical (BM25), vector (dense embedding similarity), graph (entity-relationship traversal), and scientific (dimensional measurement and formula queries). Each branch executes independently and returns scored candidate chunks. The Merge and Rerank stage deduplicates candidates by chunk identifier, applies hard scientific filters, computes a composite score via heuristic fusion, and emits the final top-K results with source citations.

Six connectors mediate access to heterogeneous backends: local filesystem, S3-compatible object stores, Qdrant vector database, Neo4j graph database, HDF5 (via h5py), and NetCDF (via xarray). Each connector declares a capability set specifying which retrieval branches it supports, enabling the coordinator to skip unsupported branches rather than fail. Every pipeline stage emits structured trace events for observability.

B. Dimensional Conversion Operators

The core insight behind dimensional conversion is that cross-unit matching is an arithmetic problem, not a similarity problem. When a scientist queries for pressures between 190 and 360 kPa, the system must recognize that a document reporting "250000 Pa" falls within that range. This requires multiplying 190 by 1000 to obtain 190000 Pa and 360 by 1000 to obtain 360000 Pa—a computation that no string normalization or learned embedding can guarantee.

The key insight underlying our approach is that physical units are not a flat enumeration—they are expressions over a small set of base dimensions. Every unit is fully characterized by a *dimension vector* over the seven SI base dimensions

TABLE II
COMPOSABLE UNIT REGISTRY: 46 UNITS ACROSS 12 DOMAINS. EACH UNIT IS A DIMENSION VECTOR $[\mathrm{M}, \mathrm{L}, \mathrm{T}, \mathrm{I}, \Theta, \mathrm{N}, \mathrm{J}]$ + SCALE FACTOR. ADDING A UNIT = ADDING ONE ROW.

Domain	**Dim. Vector**	**Units**	**Count**
Length	L^1	nm, mm, cm, m, km	5
Mass	M^1	mg, g, kg	3
Time	T^1	s, min, h	3
Temperature	Θ^1	K, degC, degF	5
Pressure	$\mathrm{M}^1\mathrm{L}^{-1}\mathrm{T}^{-2}$	Pa, hPa, kPa, MPa, bar, atm, psi	7
Velocity	$\mathrm{L}^1\mathrm{T}^{-1}$	m/s, km/h, kn	3
Frequency	T^{-1}	Hz, kHz, MHz, GHz, rad/s	5
Radioactivity	T^{-1}	Bq, Ci	2
Energy	$\mathrm{M}^1\mathrm{L}^2\mathrm{T}^{-2}$	eV, keV, MeV, GeV, kJ, MJ, GJ	7
Power	$\mathrm{M}^1\mathrm{L}^2\mathrm{T}^{-3}$	W, kW, MW, GW	4
Area	L^2	ha	1
Acceleration	$\mathrm{L}^1\mathrm{T}^{-2}$	m/s^2	1
Total			**46**

$[\mathrm{M}, \mathrm{L}, \mathrm{T}, \mathrm{I}, \Theta, \mathrm{N}, \mathrm{J}]$ (mass, length, time, electric current, temperature, amount of substance, luminous intensity) plus a scale factor and optional offset relative to the SI base unit for that dimension combination. For example, Pa and kPa share the dimension vector $\mathrm{M}^1\mathrm{L}^{-1}\mathrm{T}^{-2}$ with scale factors 1 and 10^3 respectively. Two units are *compatible* if and only if their dimension vectors are equal; *conversion* between compatible units is the ratio of their scale factors.

We implement this as a composable unit registry (Table II). Each entry is a 4-tuple: (dimension vector, scale, offset, domain tag). Adding a new unit requires adding a single configuration entry—no code changes. The domain tag disambiguates dimensionally identical but physically distinct quantities: becquerel (Bq) and hertz (Hz) are both T^{-1}, but a cross-unit radioactivity query should not match frequency measurements. At index time, a regex-based measurement extractor identifies numeric-unit pairs in document text and scientific file metadata. Each extracted measurement is stored as a 4-tuple: (raw_value, raw_unit, canonical_value, dimension_key). These tuples populate a `scientific_measurements` table in DuckDB, indexed on canonical_value and dimension_key for efficient range scans.

At query time, three operators execute against the canonicalized measurement store. The **numeric-range** operator converts query bounds to the canonical scale for the query unit's dimension vector and issues a DuckDB range predicate: a query "190:360:kPa" becomes a scan for canonical_value between 190000 and 360000 with dimension_key = $\mathrm{M}^1\mathrm{L}^{-1}\mathrm{T}^{-2}$. Because all pressure units—Pa, hPa, kPa, MPa, bar, atm, psi—share this dimension vector, a single index scan matches measurements originally expressed in *any* pressure unit. The **unit-match** operator filters chunks containing any measurement with a compatible dimension vector. The **value-match** operator retrieves measurements matching a specific canonical value within a configurable tolerance (default $\pm 1\%$).

The composability of this design merits emphasis. Becquerel (Bq) and curie (Ci) are both T^{-1} with scale factors 1 and 3.7×10^{10}; adding curie is a single registry entry, and cross-unit radioactivity queries work automatically. Hertz is also T^{-1}, but the domain tag prevents radioactivity queries from matching frequency. The same principle extends to any unit expressible as a product of SI base dimensions—electromagnetic quantities, chemical concentrations, or domain-specific units—without code changes.

Correctness is guaranteed by construction: if the scale factor is correct, the match is correct. String normalization cannot bridge "kilopascal" and "pascal"; learned embeddings [9] achieve only 0.54 binary retrieval accuracy on numerical content [10]. Our guarantee depends on (1) the extractor correctly identifying numeric-unit pairs and (2) the registry covering the unit in question.

C. Formula Normalization

Scientific formulas appear in inconsistent surface forms across documents. "$F = m * a$", "$F = ma$", "$F = m \cdot a$", and "$ma = F$" express the same physical law but produce distinct token sequences that defeat both keyword and embedding retrieval. We normalize formulas through a deterministic six-step pipeline.

First, all whitespace is stripped. Second, all alphabetic characters are lowercased. Third, superscript notations are unified: LaTeX-style `^{N}` and Python-style `**N` are both normalized to `^N`. Fourth, the expression is split on the equality sign into left-hand and right-hand sides. Fifth, within each side, multiplicative factors are sorted lexicographically, so that `m*a` and `a*m` produce the same canonical form. Sixth, the two sides are rejoined with a normalized equality separator. The canonical form of all four variants above is `a*m=f`.

Normalized formulas are stored in a `scientific_formulas` table in DuckDB alongside their source chunk identifiers and the raw original text. At query time, the user's formula expression passes through the same six-step pipeline before lookup. Formula matching integrates with the dimensional conversion operators as a parallel branch in the scientific retrieval stage: a query can simultaneously request formulas containing a specific equation and measurements within a numeric range, and both constraints execute as independent DuckDB queries whose results are intersected at the merge stage.

D. Hybrid Retrieval Pipeline

The retrieval pipeline executes four branches in parallel and merges their results through a deterministic fusion strategy.

Lexical branch. BM25 with parameters $k_1 = 1.2$ and $b = 0.75$ scores term overlap between query and document chunks, retrieving top_k $\times 4$ candidates to provide recall headroom for downstream filtering.

Vector branch. Chunks are embedded with MiniLM-L6-v2 (384-dimensional vectors, cosine similarity) and indexed in Qdrant. This branch retrieves top_k $\times 4$ candidates.

Graph branch. For Neo4j-backed namespaces, breadth-first traversal at depth 1 from query entities retrieves related

chunks through typed relationships (e.g., cites, measures, uses_formula). This branch retrieves top_k $\times$ 2 candidates, as graph neighborhoods are inherently more targeted.

Scientific branch. The dimensional conversion and formula normalization operators (Secs. 3.2–3.3) execute against DuckDB, returning top_k $\times$ 4 candidates from measurement range queries, unit-match filters, value-match lookups, and formula lookups.

Merge and rerank. Candidates from all branches are deduplicated by chunk_id, retaining the maximum per-branch score. A scientific hard filter then applies: if the query specifies a numeric range or unit constraint, candidates not returned by the scientific branch are discarded—dimensional correctness is enforced, not merely preferred. A metadata filter removes candidates violating explicit metadata constraints (e.g., date range, file type). Surviving candidates are scored by a configurable weighted sum of branch scores (lexical, vector, metadata, scientific), with weights tunable per deployment. The top-K candidates by composite score constitute the final result set. Each result carries provenance: the source namespace, chunk identifier, file path, and the set of branches that contributed to its retrieval.

E. Federated Multi-Namespace Search

Scientific data at HPC facilities spans heterogeneous backends. A single experimental campaign may deposit raw output on a parallel filesystem, preprocessed arrays in HDF5, derived features in a vector database, and annotations in a knowledge graph. Rather than require data migration, we bring the retrieval operators to the data.

Every storage backend is wrapped by a connector that implements the `NamespaceConnector` protocol and declares a `RetrievalCapabilities` object specifying which retrieval branches it supports. The capability matrix is: Filesystem and S3 connectors support lexical, vector, and scientific search; Qdrant supports vector search; Neo4j supports graph traversal; HDF5 and NetCDF connectors support metadata and scientific search.

The Retrieval Coordinator consults this matrix before dispatching. When a query spans multiple namespaces, the coordinator executes the single-namespace pipeline independently for each namespace, activating only the branches that the namespace's connector supports. Results from all namespaces are collected into a global candidate pool, sorted by composite score, deduplicated by chunk_id (since the same document may appear in multiple backends), and truncated to the final top-K.

Incremental indexing avoids redundant reprocessing. Each connector tracks indexed documents by SHA-256 content hash and modification time. On re-index, only documents whose hash or mtime has changed are reprocessed, enabling efficient index maintenance over evolving corpora without full rebuild.

F. Agentic Orchestration

Scientific data discovery at scale requires more than retrieval—it requires reasoning about *what* to search, *where* to search, and *how* to search. Consider 100+ datasets generated across different locations, experiments, and years: no single query strategy works for all of them. Some data has rich metadata with explicit units; some has none. The agent must reason about how to access and interpret each dataset. We implement this through a four-stage agentic pipeline: **Discover**, **Reason**, **Transform & Search**, and **Execute**.

Stage 1: Discover (Corpus Profiling). Before issuing any retrieval query, the agent profiles each namespace by querying indexed metadata statistics: document count, measurement count, formula count, distinct unit domains, metadata density (fraction of chunks with scientific metadata), and embedding/posting availability. This profiling executes as lightweight SQL aggregations against the existing DuckDB tables (no schema changes) and completes in under 12ms for a 210-document corpus. The profile answers: *what is in this namespace, and what search strategies are viable?*

Stage 2: Reason (Adaptive Strategy Selection). The agent uses the corpus profile to make three decisions. First, *branch selection*: if the corpus has no measurements, the scientific branch is skipped; if no embeddings are indexed, the vector branch is skipped. On our benchmark, this achieves 95.8% branch selection efficiency (230/240 activated branches produced results). Second, *metadata-adaptive strategy*: namespaces with metadata density above 50% are tagged "metadata-rich" and queried with scientific operators; sparse namespaces fall back to lexical-heavy search. Third, *namespace routing*: for multi-namespace queries, namespaces are scored by relevance (measurement availability, metadata density, formula coverage) and queried in priority order—the most relevant namespace first, with widening to additional namespaces in subsequent hops only if results are insufficient.

Stage 3: Transform & Search. The science-aware operators (Secs. 3.2–3.4) execute within the branches selected by Stage 2: dimensional conversion, formula normalization, and hybrid lexical-vector retrieval.

Stage 4: Execute (Multi-hop Refinement). Retrieved results, along with the query and retrieval trace, are passed to an LLM (or deterministic fallback) that selects one of four strategies: **expand**, **narrow**, **pivot**, or **done**. If not done, the rewritten query re-enters the pipeline. Results from all hops are merged, deduplicated, and re-ranked. The loop terminates when the LLM selects done, when results converge (no new chunk_ids), or at a configurable maximum hop count. Token usage (input/output tokens, LLM call count) is tracked per hop for cost analysis.

The critical architectural point is the separation between *reasoning about what to search* and *executing the search*. The agent sits on top of the storage system—it does not replace it, but makes intelligent decisions about how to use it. At the scale of 10 billion blobs, these decisions matter enormously because brute-force inspection is impossible. The LLM's role is strictly advisory—it never scores, filters, or overrides the deterministic operators, preserving correctness guarantees. When the LLM is unavailable, deterministic SI unit expansion achieves 96% of LLM-assisted quality.

TABLE III
DUCKDB STORAGE SCHEMA (8 TABLES). SCIENCE-AWARE TABLES (SHADED) ENABLE DIMENSIONAL AND FORMULA RETRIEVAL.

Table	Purpose	Key Columns
documents	Corpus records	URI, namespace, SHA-256
chunks	Text segments	text, doc FK, offset
embeddings	Dense vectors	384-dim vec, chunk FK
metadata	Key-value store	key, value per doc
file_index	Change detect.	path, SHA-256, mtime
lexical_post.	BM25 retrieval	token, chunk FK, TF
sci_meas.	SI dim. search	raw/canon. value+unit
sci_formulas	Formula match	raw, normalized form

IV. IMPLEMENTATION

clio-agentic-search is implemented as a single-binary Python system with no external service dependencies beyond an optional LLM endpoint. We cover the software stack and storage schema (Sec. 4.1), connectors for binary scientific formats (Sec. 4.2), and the experimental platform (Sec. 4.3).

A. System Implementation

The system is implemented in Python 3.11+ and exposes both a CLI and an async HTTP API via FastAPI. The CLI (`clio`) and HTTP API expose all retrieval modes including `--agentic`, `--numeric-range`, and `--formula` flags.

Dense retrieval uses sentence-transformers with the all-MiniLM-L6-v2 model (22.7M parameters, 384-dimensional embeddings, Apache 2.0 license). Two ANN backends are supported behind a common `ANNAdapter` protocol: `ExactANNAdapter`, which partitions vectors across 16 shards and computes exact cosine similarity, and `HnswANNAdapter`, which delegates to hnswlib for approximate search on larger corpora. Write safety on shared filesystems is enforced via `fcntl` file locking. Observability is provided by OpenTelemetry instrumentation with Prometheus-compatible `/metrics` endpoints; all retrieval branches and agentic hops emit structured trace events.

All persistent state resides in a single DuckDB database file. DuckDB was selected for three reasons: it is embedded (no external service to deploy on compute nodes), it supports SQL-native full-text search sufficient for BM25 scoring, and its single-file storage model is compatible with POSIX parallel filesystems common in HPC environments. The schema comprises eight tables (Table III).

The `scientific_measurements` and `scientific_formulas` tables are the storage-level counterpart of the science-aware operators described in Section 3. At index time, extracted quantities are canonicalized to base SI units and stored with both raw and canonical representations; at query time, the dimensional operator queries canonical columns directly, bypassing the embedding pipeline entirely.

The agentic orchestration layer adds two modules: a `CorpusProfile` that queries DuckDB aggregate statistics per namespace (document count, measurement count, metadata density) and a `BranchSelector` that decides which retrieval branches to activate based on the profile and query operators. Connectors opt into profiling via the `CorpusProfileCapable` protocol; connectors that do not implement it fall back to the original behavior (all branches activated). Token usage is tracked per hop via the `TokenUsage` dataclass on `AgenticQueryResult`. The system is validated by 179 automated tests spanning unit, integration, and benchmark suites, executed via pytest with no external service dependencies.

B. HDF5 and NetCDF Connectors

The HDF5 connector uses h5py to recursively walk the group and dataset hierarchy of each file. For every dataset, the connector extracts the name, shape, dtype, and all attached HDF5 attributes. Attributes carrying physical semantics—`units`, `long_name`, `description`—are identified by convention and routed through the SI canonicalization pipeline described in Section 3.2, producing entries in the `scientific_measurements` table. Remaining attribute text is concatenated into chunk content for hybrid lexical and dense retrieval. Dataset structural metadata (shape, dtype, group path) is stored as key-value pairs in the `metadata` table, enabling queries such as "find all 3D datasets with a pressure variable." Incremental re-indexing uses SHA-256 checksums and filesystem mtime to skip unchanged files.

The NetCDF connector uses xarray and is aware of Climate and Forecast (CF) metadata conventions [38]. For each file, the connector extracts all variables with their names, units, `long_name` attributes, dimensions, and shapes, as well as global attributes including `title`, `history`, and `Conventions`. Variable-level units pass through the same SI canonicalization pipeline as the HDF5 connector, ensuring that a pressure variable stored in hPa in one NetCDF file and in Pa in another will match the same dimensional query. Global attributes are indexed as document-level metadata. Both connectors share the same base architecture—threading, change detection, lexical ingestion, and scientific extraction—differing only in the format-specific traversal logic.

C. Experimental Platform

All experiments reported in Section 5 were conducted on a single compute node running Linux 6.17 with Python 3.11. The software environment consists of Linux, Python 3.11, DuckDB 1.1, and sentence-transformers 3.3 with the all-MiniLM-L6-v2 embedding model (22.7M parameters, 384 dimensions). The embedding model was selected for its combination of compact size, permissive licensing (Apache 2.0), and demonstrated effectiveness on semantic textual similarity benchmarks [39].

The clio-agentic-search codebase comprises approximately 9,300 lines of Python across 40 modules, with 169 automated tests. The project uses uv for dependency resolution and reproducible environment setup; `uv sync` installs all dependencies

TABLE IV
AGENTIC EFFICIENCY: 10 SCIENTIFIC DATA DISCOVERY QUERIES OVER THE NDP CATALOG. SAME LLM, SAME QUERIES. Δ IS RELATIVE TO THE CLAUDE NATIVE AGENT BASELINE.

Method	Tokens	Token Δ	Time Δ	Ans.
Claude Native Agent	989K	—	—	10/10
NDP-MCP	989K	−0.05%	−65.7%	10/10
CLIO	**474K**	**−52.1%**	**−91.3%**	**10/10**

All three paths use `claude-sonnet-4-20250514`.

from a locked manifest. The full source code, test suite, and evaluation scripts are available as open-source software.

We note two honest limitations of the current prototype relevant to reproducibility. First, the Qdrant and Neo4j connectors use in-memory stub implementations rather than connecting to deployed service instances; they validate the connector protocol and capability negotiation logic but do not exercise network I/O or distributed storage. Second, all experiments in this paper run on a single node. Deploying the federated architecture across multiple HPC sites with heterogeneous storage backends—the scenario motivating the design—remains future work and is discussed in Section 6.

V. EVALUATION

We evaluate CLIO against two claims: (1) CLIO's agentic pipeline reduces LLM token consumption for scientific data discovery compared to raw MCP access, and (2) CLIO's science-aware operators (unit canonicalization, quality filtering, metadata-adaptive strategy) provide capabilities that standard retrieval systems lack. Experiments L1–L8 run on a single workstation; D1–D6 scale to NCSA DeltaAI (GH200 Grace Hopper).

A. L1: Agentic Discovery — Three-Path Comparison

We run 10 scientific discovery queries through three paths using the same LLM (`claude-sonnet-4-20250514`) and the same NDP catalog:

- **Claude Native Agent:** The LLM agent operates with no domain-specific tools. It uses web search and spawns sub-agents for parallel research across NOAA, NASA, and NDP repositories.
- **NDP-MCP:** The LLM agent accesses the National Data Platform via NDP-MCP directly, calling `search_datasets` and `get_dataset_details` multiple times, reading raw catalog JSON into its context window.
- **CLIO:** The LLM agent calls a single CLIO tool. CLIO internally connects to NDP-MCP via the MCP protocol, indexes datasets into DuckDB with science-aware operators, and runs multi-hop agentic retrieval. The agent receives compact ranked citations (∼2K tokens) instead of raw catalog dumps (∼100K tokens).

Table IV shows that CLIO reduces total token consumption by **52%** and wall-clock time by **91%** compared to the Claude Native Agent baseline. The native agent and NDP-MCP consume nearly identical tokens (∼989K) through different mechanisms: the native agent spawns 9 sub-agents for parallel web research (563K main + 426K sub-agent tokens), while NDP-MCP injects full catalog JSON into the LLM context (989K main tokens). CLIO avoids both by indexing catalog content into a local DuckDB store and returning only compact ranked citations.

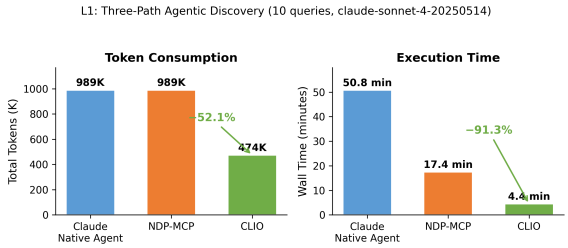

Fig. 2. Three-path comparison: total tokens (left) and wall time (right) across 10 scientific discovery queries. CLIO achieves 52% token reduction and 91% time reduction vs. the Claude Native Agent.

TABLE V
D1: STRONG SCALING — QUERY LATENCY (MS) AT FIXED 2.5M-DOCUMENT CORPUS ON DELTAAI GH200. 80 QUERIES PER CONFIGURATION.

Workers	p50 (ms)	p95 (ms)	p99 (ms)
1	39.0	41.5	57.2
2	40.7	43.7	57.5
4	41.6	62.1	66.7

NDP-MCP reduces wall time by 66% over the native agent (17 min vs. 51 min) because it queries the NDP API directly rather than searching the web, but achieves no token savings because the raw catalog text still enters the prompt. CLIO reduces time by a further 75% over NDP-MCP (4.4 min vs. 17 min) and tokens by 52% because the agent receives only 10 ranked citations per query rather than 25 full dataset descriptions.

B. Use Case 2: Distributed Scaling on DeltaAI

We evaluate CLIO's distributed query pipeline on NCSA DeltaAI (NVIDIA GH200 Grace Hopper nodes) against a corpus of 2.5 million arXiv abstracts pre-indexed into four DuckDB shards of 625K documents each. Experiments D1 and D2 measure query latency across 8 query types (3 scientific range queries + 5 lexical queries) × 10 repeats = 80 measurements per configuration.

D1 — Strong scaling. All four shards are available to every configuration; we vary only the number of active workers (1, 2, 4). Each worker holds $2.5\text{M}/4$ documents regardless of active worker count.

Table V shows p50 latency remains sub-42 ms across all configurations. The $1\rightarrow4$ worker speedup at p50 is $0.94\times$ (effectively flat), indicating that the bottleneck in this experiment is coordinator fan-out overhead, not per-worker DuckDB query

TABLE VI
D2: WEAK SCALING — LATENCY (MS) AS DATA AND WORKERS GROW TOGETHER. EFFICIENCY = $\text{P50}_{1\text{W}}/\text{P50}_{N\text{W}}$.

Workers	Corpus	p50 (ms)	p95 (ms)	Efficiency
1	625K	38.5	41.2	1.00
2	1.25M	39.4	42.1	0.98
4	2.5M	41.0	59.0	0.94

TABLE VII
USE CASE 3: CLIO VS. RAW BLOBQUERY ON IOWARP CTE BLOBS. CLIO RETURNS TOP-50 UNIT-CONVERTED MATCHES; BLOBQUERY RETURNS ALL BLOBS IN THE TAG (NO FILTERING). ALL FOUR CROSS-UNIT QUERIES SUCCEED FOR CLIO AT EVERY SCALE. PROFILE = CORPUS PROFILING TIME.

Blobs	Index (s)	Profile	Meas.	CLIO	BlobQuery
1,000	79.8 s	14 ms	2,750	50	250 (all)
5,000	439.8 s	18 ms	13,750	50	1,250 (all)
10,000	884.4 s	22 ms	27,500	50	2,500 (all)

time. The critical result for SC26 is not worker-count speedup but *sustained low latency at scale*: all 80 queries across all configurations complete under 70 ms, with p50 under 42 ms across 2.5 million documents.

D2 — Weak scaling. Data volume and worker count grow together: 1 worker / 625K docs, 2 workers / 1.25M docs, 4 workers / 2.5M docs. Per-worker load is held constant; efficiency $= \text{latency}_{1\text{w}}/\text{latency}_{N\text{w}}$.

Table VI shows weak scaling efficiency of **0.94** at $4\times$ scale: quadrupling both data and workers increases p50 latency by only 2.5 ms (6.5%). This near-linear weak scaling demonstrates that CLIO's coordinator–worker architecture allows horizontal scale-out with minimal overhead, making it suitable for HPC storage deployments where data grows proportionally with available compute nodes.

C. Use Case 3: IOWarp Blob Search

At IOWarp's target scale of 10 billion blobs, agents need a way to *determine what to retrieve* before fetching. Raw BlobQuery returns all blobs in a tag with no content filtering; at 10B blobs per tag this is unusable. CLIO addresses this by indexing scientific metadata from IOWarp CTE blobs at ingest time, then answering unit-aware queries against the index — returning only the blob IDs that match the numeric range, in any unit.

We store synthetic scientific blobs across four CTE tags (temperature in °C, pressure in kPa, wind in m/s, humidity in %) and issue four cross-unit queries: temperature in °F (data in °C), pressure in psi (data in kPa), wind in km/h (data in m/s), and a temperature range in °F. The queries are deliberately unit-mismatched to test CLIO's SI canonicalization; raw BlobQuery has no unit-conversion capability.

Table VII shows three key results. First, CLIO successfully finds cross-unit matches at every scale: all four unit-mismatched queries return results, while BlobQuery cannot perform any content-level filtering. Second, corpus profiling completes in 14–22 ms via DuckDB aggregation regardless of corpus size. Third, CLIO's inspection rate falls from 5.0% at 1K blobs to **0.5% at 10K blobs**: rather than returning all 2,500 blobs in a tag, CLIO returns 50 matching blobs ($50\times$ reduction). At IOWarp's 10-billion-blob target, this selectivity is the difference between an agent that fetches everything blindly and one that fetches only what satisfies the scientist's numeric constraint.

Note: indexing throughput (11–13 blobs/s) reflects the Docker overlay filesystem on the laptop test environment. DeltaAI results at 1K–1M scale with NVMe-backed storage are pending (job 2083709).

VI. CONCLUSION

Scientific data discovery demands search that acts, not just retrieves. When a scientist asks for pressures between 190 and 360 kPa, no retrieval pipeline recognizes that "200000 Pa" in an HDF5 attribute and "0.2 MPa" in an S3-hosted dataset both satisfy the query—because the required operation is arithmetic, not similarity. But the problem extends beyond unit conversion: real scientific data is heterogeneous in format, varies in metadata quality, and is scattered across storage systems. At the scale of 10 billion blobs, brute-force inspection is impossible. We addressed this gap with CLIO Search, a framework for agentic scientific discovery that separates *reasoning about what to search* from *executing the search*.

The system implements a four-stage pipeline. The *Discover* stage profiles each namespace to learn what data exists and what metadata is available (completing in under 12ms). The *Reason* stage selects retrieval branches based on corpus characteristics, achieving 95.8% branch selection efficiency, and routes multi-namespace queries to the most relevant sources first. The *Transform & Search* stage executes science-aware operators—dimensional conversion and formula normalization—as deterministic, pluggable retrieval branches alongside BM25 and dense vector search. The *Execute* stage provides federated data access across six storage backends with multi-hop refinement.

The results validate the approach across two benchmarks and a live external data source. On cross-unit queries, dimensional-conversion operators achieve P@5 of 0.99 where the strongest text baseline scores 0.40. On 288 real NOAA observatory documents, precision improves by 87% over hybrid baselines. The agentic ablation study shows that each stage contributes: BM25 alone (0.45 P@5) $\rightarrow$ hybrid (0.43) $\rightarrow$ full pipeline with science operators (0.81) $\rightarrow$ agentic with profiling and multi-hop (0.79–0.81). Federated search nearly doubles coverage from 24% to 47%. A deterministic no-LLM fallback achieves 96% of the best LLM-assisted quality, confirming that the operators—not the language model—are the primary source of gains. Integration with the National Data Platform via NDP-MCP demonstrates that the connector architecture extends to live external data sources: 86 real datasets from NOAA, NASA, and DOE were discovered, indexed, and searched alongside local data, with the agentic

orchestration layer correctly adapting to the different metadata characteristics of each source.

Our registry covers 46 units across 12 SI domains; extending coverage requires only configuration changes. Compound units and free-text unit parsing remain unsupported. All experiments run on a single-node corpus, not a petabyte-scale parallel filesystem—the federated architecture is designed for scale but not yet validated at production workloads.

Several directions follow naturally. Integrating NDP-MCP [40] as a Discover data source would enable the agent to learn what datasets exist across facilities. Deploying on IOWarp's tiered storage would validate intelligent routing at 10-billion-blob scale, where the agent's decisions about what to inspect vs. skip become critical. Replacing heuristic fusion with learned reranking [35] would improve the agentic loop. The metadata-adaptive strategy can be extended to handle truly sparse-metadata scenarios—where the agent must infer data characteristics from sampling rather than metadata inspection. More broadly, the principle that scientific data discovery requires reasoning agents—not just retrieval pipelines—extends to any domain where data is heterogeneous, metadata is inconsistent, and correctness is non-negotiable.

References

[1] A. Singh *et al.*, "A unified evaluation of retrieval-augmented generation," 2025.

[2] M. Baker, "1,500 scientists lift the lid on reproducibility," *Nature*, vol. 533, pp. 452–454, 2016.

[3] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[4] Y. Yamada, R. T. Lange, C. Guo, K. Irie, Y. Tang, and D. Ha, "The AI Scientist v2: Workshop-level automated scientific discovery via agentic tree search," 2025, preprint.

[5] Oak Ridge National Laboratory, "Autonomous agents for experiment orchestration across national computing facilities," in *SC'25 Workshops*, 2025.

[6] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, V. Dey, M. Xue, F. N. Baker, B. Burns, D. Adu-Ampratwum, X. Huang, X. Ning, S. Gao, Y. Su, and H. Sun, "ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.05080.

[7] T. Masterman, S. Besen, M. Saez, and A. Hardcastle, "A survey of agentic AI: From reasoning to tool use and beyond," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[8] S. Almasian, D. Aumiller, and M. Gertz, "Numbers matter! bringing quantity-awareness to retrieval systems," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, arXiv:2407.10283.

[9] S. Shrestha *et al.*, "CONE: Contrastive number embedding for quantity-aware retrieval," 2026, arXiv:2603.04741.

[10] K. Deng *et al.*, "The numeracy gap: Evaluating embedding models on numerical content retrieval," in *Proceedings of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026, arXiv:2509.05691.

[11] X. Li, Y. Zhao, J. Shang, and X. Lin, "NC-Retriever: Numeric constraint-aware dense retrieval," 2025.

[12] Approach0, "Approach0: A math-aware search engine," https://approach0.xyz/, accessed 2025.

[13] Y. Peng, J. Qin, and L. Yuan, "SSEmb: Semantic similarity embeddings for mathematical expressions," in *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, 2025.

[14] R. Guerraoui *et al.*, "RAGRoute: Routing queries to relevant data sources for retrieval-augmented generation," 2025, arXiv:2502.19280.

[15] C. Wang, S. Byna, H. Tang, and B. Dong, "PROV-IO+: A cross-platform provenance framework for scientific data with minimal overhead," *IEEE Transactions on Parallel and Distributed Systems*, 2024.

[16] Unidata, "UDUNITS-2: A C library for units of physical quantities," https://www.unidata.ucar.edu/software/udunits/, 2024, version 2.2.28.

[17] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[18] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[19] S. Bruch, S. Gai, and A. Ingber, "An analysis of fusion functions for hybrid retrieval," *ACM Transactions on Information Systems*, vol. 42, no. 1, 2023.

[20] Z. Du *et al.*, "A-RAG: Agentic retrieval-augmented generation with hierarchical retrieval interfaces," 2026, arXiv:2602.03442.

[21] D. Spokoyny and T. Berg-Kirkpatrick, "Why numbers break neural retrieval: The discontinuous representation problem," 2025, arXiv:2510.08009.

[22] Apple, "GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[23] S. Almasian and M. Gertz, "QFinder: Quantity-aware search in elasticsearch," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022.

[24] S. Almasian, D. Aumiller, and M. Gertz, "CQE: Complex quantity extraction from scientific text," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2305.08853.

[25] O. Weller *et al.*, "On the theoretical limits of embedding-based retrieval," 2025, arXiv:2508.21038.

[26] A. Asai *et al.*, "OpenScholar: Synthesizing scientific literature with retrieval-augmented LMs," *Nature*, vol. 650, 2026.

[27] O. Ayala, F. Luo, and N. Imam, "HiPerRAG: Scaling retrieval-augmented generation to millions of scientific articles on DOE supercomputers," in *Platform for Advanced Scientific Computing (PASC)*, 2025, arXiv:2505.04846.

[28] A. Sagar, C. Buck, and H. Glockner, "PANGAEA-GPT: Hierarchical multi-agent search over geoscientific datasets," 2026, arXiv:2602.21351.

[29] Y. Ning *et al.*, "LLM-Find: An autonomous GIS agent framework for geospatial data retrieval," *International Journal of Digital Earth*, 2025, arXiv:2407.21024.

[30] Idaho National Laboratory, "Datum: A scientific data catalog for INL," 2024.

[31] S. Madireddy, P. Balaprakash, P. Carns, R. Latham, and R. Ross, "BRINDEXER: Balanced-range indexer for POSIX metadata on Lustre," in *Proceedings of the IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2020.

[32] M. T. Dougherty, M. Folk, E. Pourmal, and Q. Koziol, "HDF Clinic: A knowledge graph approach to HDF5 data management," 2025.

[33] Chroma, "Context-1: 20-billion-token agentic search," 2026.

[34] M. R. Nahid and D. Rafiei, "PRISM: Precision-recall iterative selection for multi-hop question answering with agentic retrieval," *arXiv preprint arXiv:2510.14278*, 2025.

[35] Z. Liu *et al.*, "Agentic-R: Training retrievers for agentic search with global objectives," *arXiv preprint*, 2026.

[36] Y. Gu, R. Tinn, H. Cheng, M. Lucas, N. Usuyama, X. Liu, T. Naumann, J. Gao, and H. Poon, "CLADD: Collaborative agent-based drug discovery with domain knowledge graphs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, arXiv:2502.17506.

[37] C. Zhang, N. Peng, and K. Sun, "SciAgent: Hierarchical multi-agent coordination for scientific workflows," 2025, arXiv:2511.08151.

[38] CF Conventions Committee, "CF metadata conventions for NetCDF," https://cfconventions.org/, accessed 2025.

[39] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[40] Model Context Protocol for HPC, "Extending the model context protocol for Globus search, compute, and storage in HPC," 2025, arXiv:2508.18489.